Insper

Megadados

Programação funcional

Marcel Parolin Jackowski mjack@insper.edu.br
Maciel Vidal macielcv@insper.edu.br
Fábio Ayres fabioja@insper.edu.br

Total pago por CPF, em Python

Especificação

```
get_total_pgto(cpfs, pagamentos)
```

- `cpfs`: lista de CPFs
- `pagamentos`: lista de dicionários com id, cpf e valor
- **Devolve** um dicionário que associa cada CPF ao valor total que ele pagou (0 se não pagou nada)

Exemplo

```
cpfs = ["111", "222", "333"]
pagamentos = [
  {"id": 1, "cpf": "111…", "valor": 50},
  {"id": 2, "cpf": "222…", "valor": 30},
  {"id": 3, "cpf": "111…", "valor": 20},
]

get_total_pgto(cpfs, pagamentos)
# {"111…": 70, "222": 30, "333": 0}
```

Total pago por CPF, em SQL

Dada a relação abaixo, escreva uma query que devolve o valor total pago por CPF.

pagamentos		
id	**cpf**	**valor**
INT · PK	VARCHAR(11)	DECIMAL(10,2)

Guarde esta pergunta

Quando terminar o Python e o SQL, compare as duas soluções: qual delas diz o que você quer? Qual diz como calcular?

Paradigmas de programação

Um paradigma é um conjunto de abstrações úteis para conceber, projetar e implementar sistemas computacionais.

Procedural

programa como sequência de comandos e procedimentos

Orientada a objetos

objetos que reúnem dados e comportamento

Orientada a eventos

o programa reage a eventos: cliques, mensagens

Funcional

foco de hoje

programa como composição de funções

Reativa

fluxos de dados que propagam mudanças

...e outros

cada um com suas abstrações e seus usos

O que significa “computar”?



Computar é executar um procedimento finito e mecânico (um algoritmo) que transforma uma entrada em uma saída.

Finito: descrito por um número finito de instruções.

Mecânico: cada passo é bem definido e dispensa intuição.

Mas o que conta, exatamente, como “procedimento mecânico”? Em 1930, ninguém sabia dizer com precisão...

Duas definições de computação (1936)

Alonzo Church

Cálculo lambda

Computar é aplicar e combinar funções, reescrevendo expressões.

Raiz das linguagens funcionais

≡

Alan Turing

Máquinas de Turing

Computar é seguir regras simples que leem e escrevem símbolos numa fita.

Modelo de máquina, próximo do que um computador faz

Modelos muito diferentes, e ainda assim calculam exatamente as mesmas funções.

Função computável

Intuitivamente

Uma função $f : \mathbb{N} \to \mathbb{N}$ é computável se existe um algoritmo que, para cada n, calcula f(n) em tempo finito.

Exemplo: dado n, devolver o n-ésimo número primo.

Formalmente (1936)

f é computável ⇔ existe uma máquina de Turing que a calcula ⇔ existe um termo do cálculo lambda que a calcula.

Tese de Church-Turing: as duas definições capturam a noção intuitiva.

Cálculo lambda: três tipos de termo

O cálculo lambda representa lógica, números e dados como **funções**. Interessa mais combinar funções do que aplicá-las a dados concretos.

x

Variável

Um nome, que se refere a um parâmetro.

Em Python:

```
x
```

$\lambda x.\ M$

Abstração

Cria uma função com parâmetro x e corpo M.

Em Python:

```
lambda x: M
```

$M\ N$

Aplicação

Aplica a função M ao argumento N.

Em Python:

```
M(N)
```

Todo programa do cálculo lambda é construído só com essas três formas.

Regra de cálculo: β-redução

$$(\lambda x.\ M)\ N \ \rightarrow\ M[x := N]$$

Calcular é trocar o parâmetro x pelo argumento N dentro do corpo M. M[x := N] lê-se “M com x trocado por N”.

Um passo, com números

$$(\lambda x.\ x + 1)\ 10 \ \rightarrow\ 10 + 1$$

Aqui M é x + 1 e N é 10.

Convenções de escrita

- **Aplicação associa à esquerda:** f a b = (f a) b
- **O corpo vai o mais à direita possível:**
  - λx. λy. x + y = λx. (λy. (x + y))

Exemplos: identidade e “soma 1”

$\lambda x.\ x$

Função identidade

Devolve o próprio argumento.

$(\lambda x.\ x)\ 7$

$\rightarrow 7$

Em Python: lambda x: x

$\lambda x.\ x + 1$

Função “soma 1”

Aplicada ao valor 10, resulta em 11.

$(\lambda x.\ x + 1)\ 10$

$\rightarrow 10 + 1$

$\rightarrow 11$

Em Python: lambda x: x + 1

Funções que devolvem funções

$\lambda x.\ \lambda y.\ x + y$ recebe x e devolve outra função, $\lambda y.\ x + y$.

Aplicação parcial: fornecer só o primeiro argumento. $(\lambda x.\ \lambda y.\ x + y)\ 10$ resulta em $\lambda y.\ 10 + y$, a função "soma 10".

Currying: escrever uma função de vários argumentos como funções de um argumento que devolvem funções.

Aplicando a 10 e depois a 20

$((\lambda x.\ \lambda y.\ x + y)\ 10)\ 20$

$\rightarrow (\lambda y.\ 10 + y)\ 20$ — *x := 10*

$\rightarrow 10 + 20$ — *y := 20*

$\rightarrow \mathbf{30}$ — *aritmética*

Booleanos como funções

TRUE ≡ λx. λy. x

Recebe dois valores e escolhe o primeiro.

FALSE ≡ λx. λy. y

Recebe dois valores e escolhe o segundo.

IFTHENELSE ≡ λv. λp. λq. v p q

O booleano v escolhe entre p e q.

Atividade

Será que IFTHENELSE TRUE a b = a e IFTHENELSE FALSE a b = b?

Um booleano é uma escolha entre duas opções.

Operadores lógicos

AND

$$\equiv \lambda p.\ \lambda q.\ p\ q\ p$$

Se p é TRUE, devolve q. Se p é FALSE, devolve p, que já é FALSE.

OR

$$\equiv \lambda p.\ \lambda q.\ p\ p\ q$$

Se p é TRUE, devolve p, que já é TRUE. Se p é FALSE, devolve q.

NOT

$$\equiv \lambda p.\ p\ \mathrm{FALSE}\ \mathrm{TRUE}$$

Se p é TRUE, devolve FALSE. Se p é FALSE, devolve TRUE.

Atividade: prove, por β-redução, que

$$\mathrm{NOT\ TRUE} = \mathrm{FALSE}$$

$$\mathrm{AND\ TRUE\ FALSE} = \mathrm{FALSE}$$

$$\mathrm{OR\ TRUE\ FALSE} = \mathrm{TRUE}$$

Gabarito no apêndice.

Números como funções

0 $0 \equiv \lambda f.\ \lambda x.\ x$

1 $1 \equiv \lambda f.\ \lambda x.\ f\ x$

2 $2 \equiv \lambda f.\ \lambda x.\ f\ (f\ x)$

3 $3 \equiv \lambda f.\ \lambda x.\ f\ (f\ (f\ x))$

O número n é a função que aplica f, n vezes, a x.

PLUS

$$\equiv \lambda m.\ \lambda n.\ \lambda f.\ \lambda x.\ m\ f\ (n\ f\ x)$$

Aplica f n vezes a x e, depois, mais m vezes.

Atividade

Prove que PLUS 1 2 = 3.

Gabarito no apêndice.

Lições do cálculo lambda: funções

Funções puras

Mesma entrada, mesma saída, sem efeitos colaterais: a função não lê nem modifica estado externo.

```
# pura
def dobro(x):
    return 2 * x

# impura
def soma(x):
    global total
    total += x
```

Funções de primeira classe

Funções são valores: podem ser guardadas em variáveis, passadas como argumento e devolvidas.

```
f = dobro
f(4)            # 8
g = f           # mesma
                # função
```

Funções de ordem superior

Funções que recebem funções como argumento, devolvem funções, ou as duas coisas.

```
def duas_vezes(f, x):
    return f(f(x))

duas_vezes(dobro, 3)
# 12
```

Lições do cálculo lambda: execução

Aplicação parcial

Fornecer só alguns dos argumentos de uma função e obter uma nova função que espera os restantes.

$$(\lambda x.\, \lambda y.\, x + y)\ 10$$

$$\rightarrow \lambda y.\, 10 + y$$

Recursão no lugar de laços

Sem variável que muda, não há contador de laço. Para repetir, a função chama a si mesma.

```
def fat(n):
    if n == 0:
        return 1
    return n * fat(n - 1)
```

Ausência de estado

Nada é modificado depois de criado: valores apenas passam de uma função para outra. Sem atribuições.

```
n = n + 1
# atribuição: não existe
# no cálculo lambda
```

O que é programação funcional?

Programação funcional é um estilo em que o programa é construído combinando funções puras, de primeira classe e de ordem superior, sem estado que muda. Inspirada no cálculo lambda e na teoria de categorias.

Problema: somar os quadrados dos números pares de uma lista nums.

Imperativo: descreve como calcular

```
total = 0
for x in nums:
    if x % 2 == 0:
        total += x * x
```

Funcional: descreve o que calcular

```
sum(x * x
    for x in nums
    if x % 2 == 0)
```

Robustez
menos efeitos colaterais

Clareza
funções pequenas e combináveis

Foco no "o quê"
não no "como"

Linguagens e estilo funcional

Funcionais	Mistas	Imperativas
Feitas para o estilo funcional.	Têm mecanismos nativos que ajudam a programar de forma funcional.	Sem mecanismos nativos para programação funcional.
Haskell	**Python**	**C**
Lisp	**Scala**	**Java (antes do Java 8)**
...	**...**	**...**

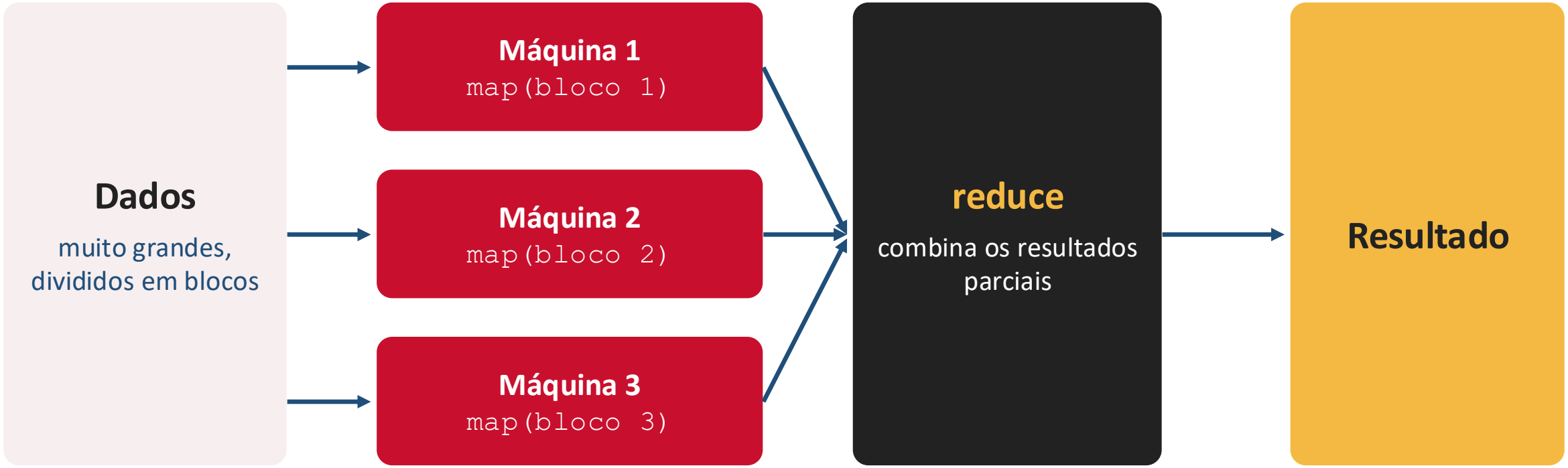
Em todas linguagens modernas é possível adotar um estilo funcional de programação

Por que funcional combina com big data

1	**Descreve o que calcular, sem dizer como iterar**	Fácil de paralelizar: cada pedaço dos dados pode ser processado por uma máquina diferente.
2	**Usa funções puras**	Robustez: se uma máquina cai, só o bloco dela é recalculado, sem efeitos colaterais a desfazer.
3	**Não tem estado global**	Facilita a memória distribuída: não há nada compartilhado que precise ser sincronizado.

Características ideais para computação distribuída de grandes massas de dados.

Map-reduce em paralelo



A mesma função pura roda em cada bloco. Se uma máquina falhar, só o bloco dela é recalculado.

Hadoop MapReduce Apache Spark

Lab: programação funcional em Python

1 **Funções de primeira classe**

```
f = dobro
```

2 **lambda**

```
lambda x: x + 1
```

3 **Closures e aplicação parcial**

```
partial(f, exp=2)
```

4 **List comprehensions**

```
[x*x for x in xs]
```

5 **map**

```
map(f, xs)
```

6 **filter**

```
filter(p, xs)
```

7 **reduce**

```
reduce(f, xs, 0)
```

Funções são valores; lambda cria funções

Função de primeira classe: pode ser atribuída a uma variável, passada como argumento e devolvida por outra função.

lambda: cria uma função anônima a partir de uma única expressão.

```
def dobro(x):
    return 2 * x

f = dobro              # função em variável
def aplica(g, v):      # função como argumento
    return g(v)
aplica(f, 5)           # 10

soma1 = lambda x: x + 1
soma1(10)              # 11
(lambda x: x * x)(7)   # 49
```

Closures e aplicação parcial

Closure: função interna que "lembra" as variáveis do escopo onde foi criada, mesmo depois que a função externa terminou.

Aplicação parcial: fixar alguns argumentos de uma função e obter uma nova função com os restantes (`functools.partial`).

`soma10` é a versão em Python de $(\lambda x.\, \lambda y.\, x + y)\ 10$

```
def somador(x):
    def soma(y):
        return x + y       # x vem de fora
    return soma

soma10 = somador(10)
soma10(20)                 # 30

from functools import partial
def potencia(base, exp):
    return base ** exp
quadrado = partial(potencia, exp=2)
quadrado(7)                # 49
```

List comprehensions

Uma list comprehension constrói uma lista a partir de outra, declarando o que entra e o que fica.

`expressão`
o que vai na nova lista

`for item in iterável`
de onde vêm os itens

`if condição`
opcional: quais itens ficam

```
nums = [1, 2, 3, 4, 5, 6]

[x * x for x in nums]
# [1, 4, 9, 16, 25, 36]

[x * x for x in nums if x % 2 == 0]
# [4, 16, 36]
```

map, filter e reduce

`map(f, itens)`

Aplica f a cada item e devolve os resultados.

`filter(p, itens)`

Mantém só os itens para os quais p(item) é verdadeiro.

`reduce(f, itens, inicial)`

Combina os itens dois a dois, acumulando um único valor.

```
from functools import reduce
nums = [1, 2, 3, 4, 5]

list(map(lambda x: x * 2, nums))           # [2, 4, 6, 8, 10]
list(filter(lambda x: x % 2 == 1, nums))   # [1, 3, 5]
reduce(lambda acc, x: acc + x, nums, 0)    # 15
```

map e filter devolvem iteradores: use list(...) para ver o resultado.

Resumo

Insper

1. Computável é o que uma máquina de Turing calcula, e isso coincide com o cálculo lambda: a tese de Church-Turing.
2. O cálculo lambda ensina funções puras, de primeira classe e de ordem superior, e a vida sem estado.
3. Programação funcional adota esses princípios e descreve o que calcular, não como.
4. Funções puras e sem estado global são ideais para dados distribuídos: daí MapReduce e Spark.
5. Em Python: lambda, closures, partial, comprehensions, map, filter e reduce.

Aquecimento

Python (imperativo)

```
def get_total_pgto(cpfs, pagamentos):
    totais = {cpf: 0 for cpf in cpfs}
    for p in pagamentos:
        if p["cpf"] in totais:
            totais[p["cpf"]] += p["valor"]
    return totais
```

SQL

```
SELECT cpf,
       SUM(valor) AS total_pago
FROM pagamentos
GROUP BY cpf;
```

Atenção: um CPF sem nenhum pagamento não aparece no resultado do SQL. A função Python devolve 0 para ele.

Versão funcional (lab)

```
from functools import reduce

def get_total_pgto(cpfs, pagamentos):
    def total(cpf):
        valores = map(lambda p: p["valor"],
                      filter(lambda p: p["cpf"] == cpf,
                             pagamentos))
        return reduce(lambda a, b: a + b, valores, 0)
    return {cpf: total(cpf) for cpf in cpfs}
```

O que repare

Nenhuma variável é modificada: cada total é uma função pura de (cpf, pagamentos).

Custo: percorre pagamentos uma vez por CPF. A versão imperativa percorre uma vez só.

Ganho: cada CPF é independente dos outros e pode ir para outra máquina.

IFTHENELSE funciona

Com TRUE

IFTHENELSE TRUE a b	*início*
→ (λp. λq. TRUE p q) a b	*v := TRUE*
→ (λq. TRUE a q) b	*p := a*
→ TRUE a b	*q := b*
→ (λy. a) b	*TRUE = λx.λy.x, x := a*
→ a	*y := b (não aparece)*

Com FALSE

IFTHENELSE FALSE a b
→ FALSE a b
→ (λy. y) b
→ b

Os primeiros passos são os mesmos; FALSE = λx.λy.y escolhe o segundo.

NOT, AND e OR

NOT TRUE = FALSE	AND TRUE FALSE = FALSE	OR TRUE FALSE = TRUE
NOT TRUE	AND TRUE FALSE	OR TRUE FALSE
→ TRUE FALSE TRUE	→ (λq. TRUE q TRUE) FALSE	→ (λq. TRUE TRUE q) FALSE
→ (λy. FALSE) TRUE	→ TRUE FALSE TRUE	→ TRUE TRUE FALSE
→ FALSE	→ (λy. FALSE) TRUE	→ (λy. TRUE) FALSE
	→ FALSE	**→ TRUE**

Sempre que aparece TRUE a b, o resultado é a: TRUE escolhe o primeiro.

PLUS 1 2 = 3

PLUS 1 2	*PLUS, 1 e 2 são abreviações*
→ (λn. λf. λx. 1 f (n f x)) 2	*m := 1*
→ λf. λx. 1 f (2 f x)	*n := 2*
→ λf. λx. f (2 f x)	*1 g y → g y (1 = λf. λx. f x)*
→ λf. λx. f (f (f x))	*2 f x → f (f x)*
= 3	*3 = λf. λx. f (f (f x))*